INTERNSHIP & INTERN ROLE

Technical Assessments

Compiled Reference Document

CONTENTS

Track 1 — Full-Stack Developer Internship Assessment

The objective of this take-home assignment is to assess your ability to design, implement, and organize a full-stack application. We are less interested in a pixel-perfect UI and more focused on how you think, structure your code, and make engineering decisions.

Duration: 1–2 days

Tech Stack

- Next.js
- Node + Express.js
- MongoDB

Product Requirements

Build a simplified project management tool, similar in concept to Trello or Jira.

1. Authentication

- Implement user registration, login, and logout.
- Use JWT for session management and authorization.
- Passwords must be securely hashed prior to storage.

2. Dashboards & Boards

- Once authenticated, users arrive at a dashboard listing their project boards (e.g., "Engineering", "Sprint", "Marketing Tasks").
- Users can create, open, and delete project boards.

3. Columns (Lists)

- Within a board, users can create columns to represent stages of work (e.g., "Todo", "In Progress", "Done").
- Users must be able to create, rename, and delete columns.

4. Task Cards

- Columns contain individual task cards.
- Cards must include at least: Title, Description, and Status. Due dates are optional.
- Users can create, edit, delete, and move cards between columns.

Note: Drag-and-drop functionality is required.

API & Database

- **RESTful API:** Design and document your own REST API using Express. You are free to structure routes as you see fit.
- **Schema Design:** Design a logical MongoDB schema. Ex: users, boards, cards, columns (todo, in-progress etc), cards (title, description etc).

Frontend & UX

- **Component Quality:** We look for clean, modular Next.js components, logical folder structure, and effective state management.
- **Resilience:** Implement proper loading states and error boundaries. A functional, clean UI is preferred over a complex, buggy one.

Validation & Error Handling

Ensure the application handles edge cases gracefully, providing clear feedback to the user and avoiding server crashes. Areas of focus include:

- Invalid login credentials or unauthorized API access.
- Insecure Direct Object Reference (IDOR) — e.g., userA should not be able to access any resource of userB.
- Missing resources (e.g., requesting a board that doesn't exist).
- Input validation (e.g., preventing empty titles, minimum and maximum input, malformed dates).

Note: The above listed are the mandatory requirements. You are free to show your creativity in adding additional meaningful features.

AI Usage

AI coding assistants are fully permitted (including ChatGPT, Claude, GitHub Copilot, Cursor, etc.). However, you must deeply understand the code you submit.

During the follow-up interview, you will be asked to walk through your architecture, explain specific implementation details, and justify the approaches generated by AI.

Bonus Objectives

- **CI/CD pipeline:** Create a GitHub Actions workflow that executes on PR merge. It should install dependencies, build both frontend and backend code, and run the docker compose while injecting env variables from repo secrets.

Note: The objective is to ensure the code builds properly and docker-compose containers actually work.

Deliverables & Submission

Please submit a link to a public GitHub repository containing your source code. Ensure your repository maintains good hygiene (do not commit node_modules, build artifacts, or secret .env files).

Your repository MUST include:

- A thorough README.md with clear setup/run instructions.
- An explanation of any architectural assumptions or trade-offs you made.
- A working docker-compose file that includes frontend, backend, mongo services.
- An .env.example file detailing required environment variables.

Good luck, and have fun building!

Track 2 — Documentation Chatbot (RAG-based)

Duration: 1–2 days

Objective

Build an AI-powered Documentation Chatbot that can answer questions from any uploaded documentation using Retrieval-Augmented Generation (RAG).

Unlike a generic chatbot, the application should answer questions strictly based on the provided documentation and always reference where the information came from.

Problem Statement

Users often need to search through lengthy documentation to find answers. Your task is to build a chatbot that allows users to upload documentation and ask questions in natural language. The chatbot should retrieve the most relevant information from the documentation and generate an accurate answer.

Functional Requirements

Document Upload — The application should support uploading one or more documents. Supported formats:

- PDF
- HTML
- Markdown (.md)
- DOCX

Document Processing — After upload:

- Extract text from the document.
- Split the content into chunks.
- Generate embeddings.
- Store embeddings in a vector database.

You may use any vector database such as:

- ChromaDB
- FAISS
- Pinecone
- Qdrant
- Weaviate
- Any equivalent solution

Question Answering — The user should be able to ask questions about the uploaded documentation. The chatbot should:

- Retrieve relevant document chunks.
- Generate an answer using an LLM.
- Restrict responses to the uploaded documentation.

- If the answer cannot be found, clearly state that the information is not available in the provided documentation.

Response Format — Each response should include:

1. Answer
2. Source(s): document name, page number (if available), section or heading (if available)
3. Related Pages / Sections: display other relevant documentation chunks.
4. Suggested Follow-up Questions: generate 3–5 related questions that the user may ask next.

Example

Answer: JWT tokens expire after the configured expiration time.

Source: Authentication Guide, Page 12

Related Pages: Authentication Overview; Token Refresh Flow

Suggested Questions: How do refresh tokens work? How can I revoke a token? How is JWT validated?

Technical Requirements

The intern may choose any suitable technology stack. Suggested options:

- Python (FastAPI / Flask)
- Node.js (Express)
- LangChain / LlamaIndex (optional)
- OpenAI or any compatible LLM
- Any embedding model
- Any vector database

Deliverables

- Source code
- README with setup instructions
- Sample documents for testing
- A short explanation of the RAG pipeline used
- Demonstration of at least 5 successful queries

Evaluation Criteria

- Correctness of answers
- Quality of document retrieval
- Source attribution
- Code quality and structure
- UI/UX (optional but appreciated)
- Documentation

Track 3 — Scheduled AI News Digest

Duration: 1–2 days

Objective

Build an automated service that fetches the latest AI news from one or more sources, summarizes each article using an LLM, categorizes the news, and generates a daily AI news digest. The goal is to create a service that could be used internally by a team to stay updated on AI developments.

Problem Statement

Instead of manually browsing multiple websites every day, build a service that automatically collects AI news, processes it, and presents a concise daily summary.

Functional Requirements

News Collection — Fetch AI-related news from one or more public sources. Possible sources include:

- RSS feeds
- News APIs
- AI blogs
- Official company blogs
- Technology news websites

Data Extraction — For each article collect:

- Title
- Publication date
- Source
- URL
- Full article (or available content)

AI Summarization — Use an LLM to generate:

- A concise summary (2–5 sentences)
- Key takeaways
- Why the news matters

Categorization — Automatically classify news into categories such as:

- LLMs
- Research
- Open Source
- Product Launches
- Funding
- Robotics
- AI Tools

- Regulation
- Infrastructure
- Other

Daily Digest Generation — Generate a digest containing:

- Date
- Top headlines
- Article summaries
- Categories
- Links to original articles

Example

AI Daily Digest — Date: 15 July 2026

1. OpenAI releases… Category: LLMs Summary: … Key Takeaways: … Source: https://…

2. Anthropic announces… …

Scheduling

The application should automatically generate the digest on a schedule. Suggested schedule:

- Every day at 8:00 AM
- Or every 24 hours

Use any scheduling library or cron-based solution.

Bonus Features (Optional)

- Email the digest automatically
- Export as Markdown, HTML, or PDF
- Store previous digests in a database
- Remove duplicate articles
- Generate a "Top Story of the Day"
- Basic web dashboard to view historical digests
- Tag articles by companies (e.g., OpenAI, Anthropic, Google, Meta)

Technical Requirements

The intern may choose any suitable technology stack. Suggested options:

- Python
- Node.js
- FastAPI / Express
- Cron Jobs
- APScheduler
- LangChain (optional)
- OpenAI or any compatible LLM

- SQLite / PostgreSQL / MongoDB (optional)

Deliverables

- Source code
- README with setup instructions
- Configuration for news sources
- Sample generated digest
- Scheduler implementation
- Instructions for running the service locally

Evaluation Criteria

- Correctness of news collection
- Quality of AI-generated summaries
- Categorization accuracy
- Reliability of scheduling
- Code quality and maintainability
- Documentation
- Overall usability

Track 4 — Signal Triage Pipeline

Primary Skill Focus: ML/data evaluation, multi-method comparison, escalation logic

Objective

Design and build a system that classifies and prioritises a batch of incoming text messages (e.g., customer support tickets, or any text dataset you construct or source) using more than one classification method, and automatically escalates low-confidence or disagreeing cases for human review.

Task Description

- Assemble or generate a dataset of at least 80–100 short text messages spanning several categories (e.g., billing issue, bug report, feature request, spam, urgent complaint).
- Implement at least two distinct classification approaches (for example: a simple rule/keyword-based classifier, an embedding-similarity classifier, and/or a zero-shot LLM prompt classifier).
- Design a confidence and disagreement scoring mechanism: when methods disagree, or confidence falls below a threshold you define, the item should be routed to a “human review” queue instead of being auto-labelled.
- Produce a summary report comparing method accuracy, the escalation rate produced by your thresholding logic, and your reasoning for where you set the threshold.

Deliverables

- Complete, runnable codebase (script or small app).
- Sample dataset used (or generation method, if synthetic).
- Short report (500–800 words) covering: method comparison, escalation logic and threshold rationale, and what you would improve with more time.
- requirements.txt or equivalent dependency list.

Track 5 — Live Discussion Monitor & Digest

Primary Skill Focus: Real-time ingestion, classification, prioritisation, agentic summarisation

Objective

Build a pipeline that continuously ingests posts from a public discussion source, classifies and deduplicates them, scores their relevance/urgency, and produces a prioritised digest — with an optional alert trigger for high-priority items.

Task Description

- Choose a public, authentication-light source (e.g., Reddit API, Hacker News API, or an RSS aggregator) and a topic scope of your choice.
- Build an ingestion step that can run repeatedly (poll or scheduled) and pull new items.
- Classify or tag each item by subtopic, and use an LLM or prompt-based approach to score relevance and urgency.
- Deduplicate near-identical or repost items before they reach the digest.
- Generate a prioritised digest (top N items) in a clean, readable output format (PDF, Markdown, or HTML all acceptable).
- Optionally: trigger a simple alert (console log, webhook, or file write is sufficient — no need for real WhatsApp/SMS integration) for items above an urgency threshold you define.

Deliverables

- Complete, runnable codebase.
- One sample digest output generated by a live or simulated run.
- Prompt documentation: list the exact prompts used for classification/scoring, with rationale.
- Short write-up (500–800 words): architecture, how you would handle rate limits and spam at scale, and design trade-offs made.
- requirements.txt or equivalent.

Track 6 — Resilient File Sync Service

Primary Skill Focus: Backend/systems engineering with an applied AI component: uploads, integrity, access control

Objective

Build a small client-and-server tool that syncs a folder of arbitrary files to a server reliably — handling interrupted uploads, avoiding duplicate storage, enforcing basic access control on shared files, and using an AI/LLM component to add intelligence to the ingestion pipeline.

Task Description

- Implement chunked file upload from client to server.
- Support pause/resume: simulate a network interruption mid-upload and demonstrate that the system resumes from the last completed chunk without corrupting the file or re-uploading completed data.
- Implement hash-based deduplication so identical files are not stored twice.
- Implement a minimal role system (e.g., owner vs. viewer) with expiring, signed links for shared access to a file.
- Integrate an AI/LLM-based classification step that automatically tags each uploaded file (e.g., file type, likely content category, sensitivity level) once the upload completes, and flags anomalous or suspicious files (e.g., mismatched extensions, unusually structured content) for review.
- Optionally: use an LLM to auto-generate a short human-readable description or summary of each uploaded file's contents, stored alongside its metadata.
- Clearly document how chunk-tracking/idempotency is handled, how signed links are generated, validated, and expired, and how the AI classification step is invoked and its outputs are used.

Deliverables

- Complete, runnable client + server codebase.
- A short demonstration (screen recording or clear step-by-step text walkthrough) showing an interrupted upload resuming correctly.
- Documentation of the AI/LLM classification step: model or prompt used, sample inputs/outputs, and rationale for design choices.
- Short write-up (500–800 words): chunking/idempotency approach, signed-link design, the AI classification component, and known limitations.
- requirements.txt / package manifest as applicable.

Shared Requirements & Instructions (Tracks 4–6)

The following requirements, submission instructions, evaluation criteria, and guidance are identical across Tracks 4, 5, and 6.

Technical Requirements & Constraints

Primary Language	Python 3.9+ (or a language you state a clear rationale for)
Duration	1–2 days from receipt of this document
AI/LLM Use	Any commercial or open-source model — justify your choice
Error Handling	Graceful fallback on failures; basic logging required
Code Quality	Clean, modular, with comments on key logic
Credentials	Use environment variables; never hardcode API keys

Submission Instructions

- Submit as a compressed archive (.zip) or a linked private repository (GitHub/GitLab) with access granted to the evaluating team.
- Include all files necessary to run the system from scratch.
- If using paid APIs, include instructions for substituting free-tier alternatives so evaluators can test the system.
- A short screen recording (2–5 minutes) or a clear text walkthrough of a complete run is encouraged but not mandatory.

Evaluation Criteria

Submissions will be evaluated on: quality and correctness of the solution, code clarity and structure, quality of the written reasoning/documentation, and overall engineering judgment shown in the trade-offs made.

Tips & Guidance for Candidates

- Think system-first, code-second. A well-designed, modular solution is more impressive than a single-file script.
- Prioritise robustness over completeness. A system that handles errors gracefully on a smaller scope is stronger than a brittle system attempting everything.
- Document your reasoning, not just your code — we care as much about how you think through trade-offs as about the final output.
- Use of open-source models to avoid API costs is perfectly acceptable and will not be penalised.